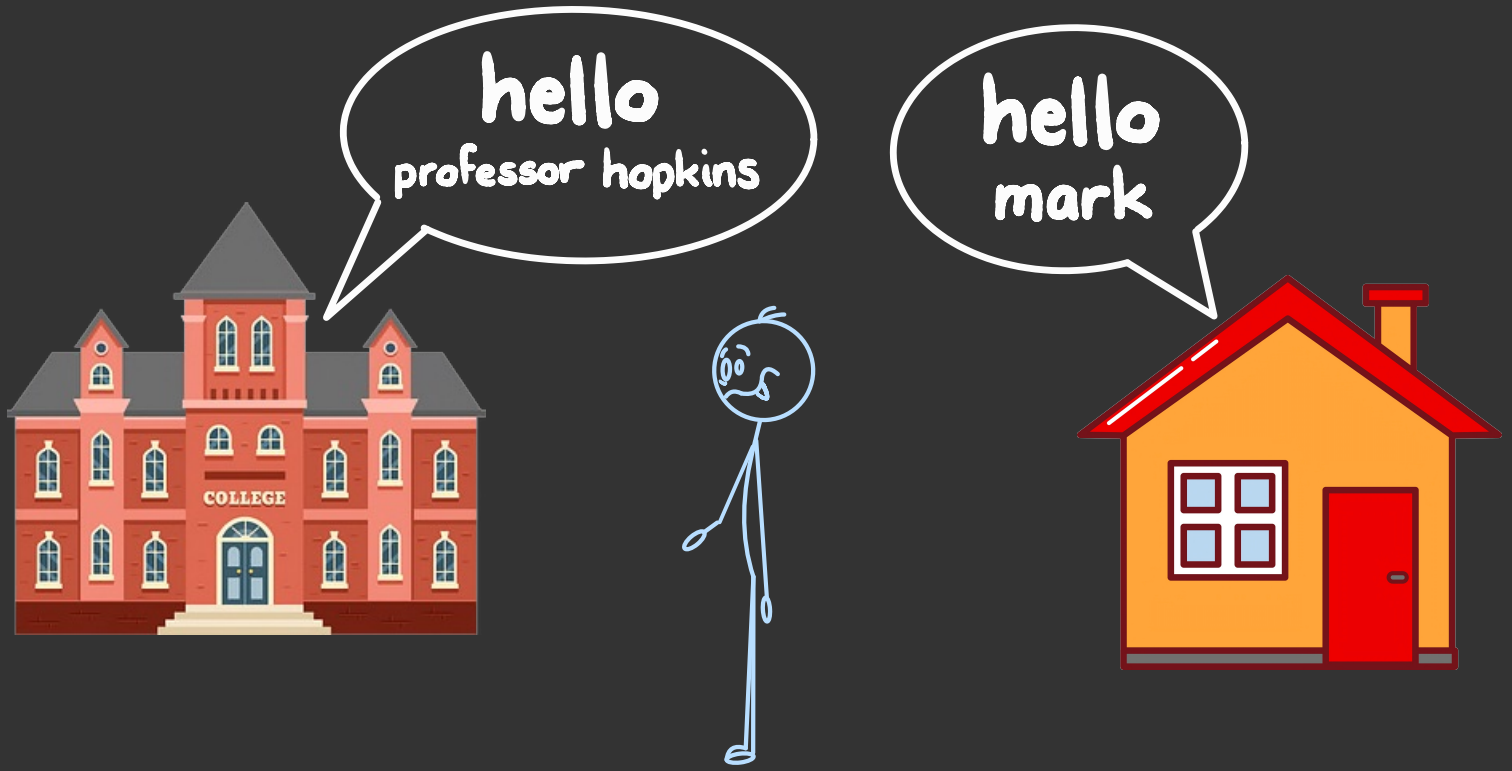
mutability
and tuples

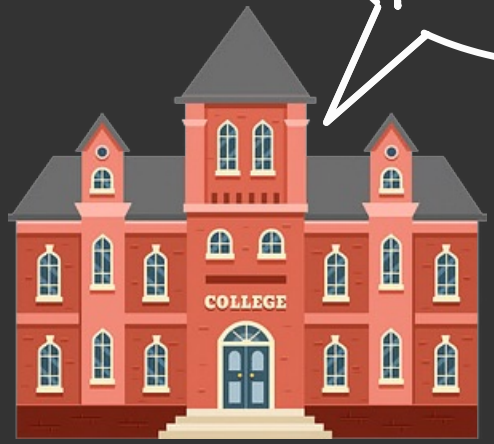
CSCI 134



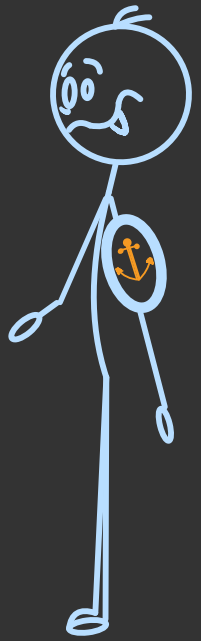
sometimes the same thing is known

by multiple names

nice tattoo professor hopkins



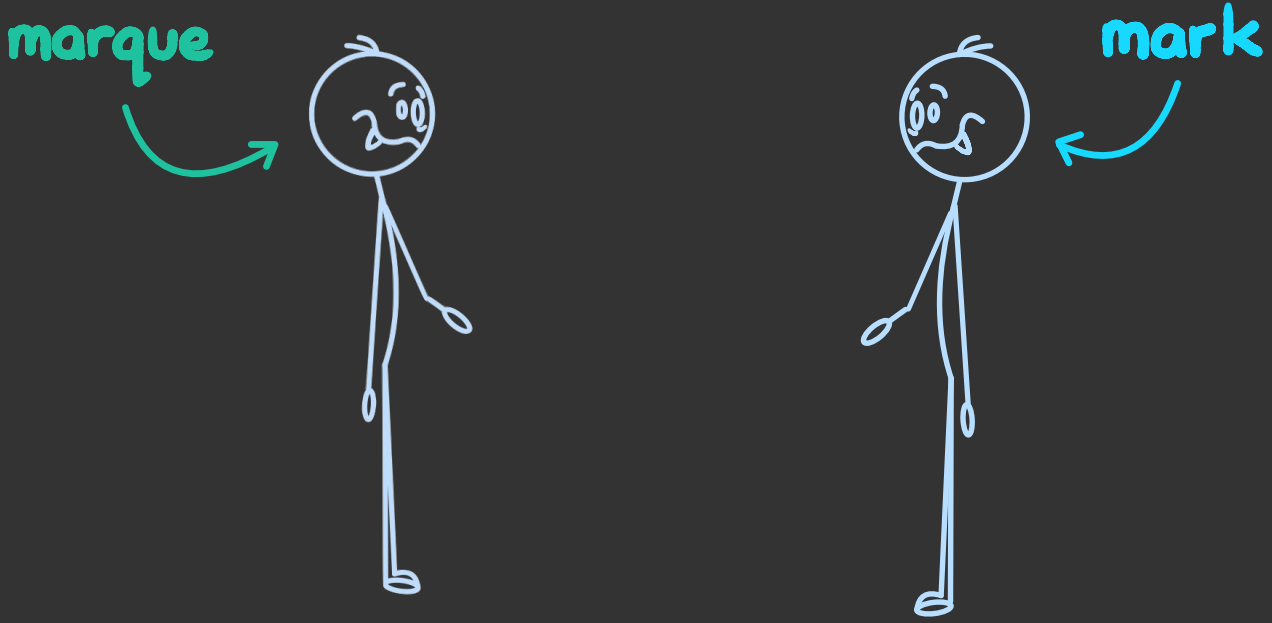
if i get a tattoo, then the people who refer to me as professor hopkins will notice the tattoo



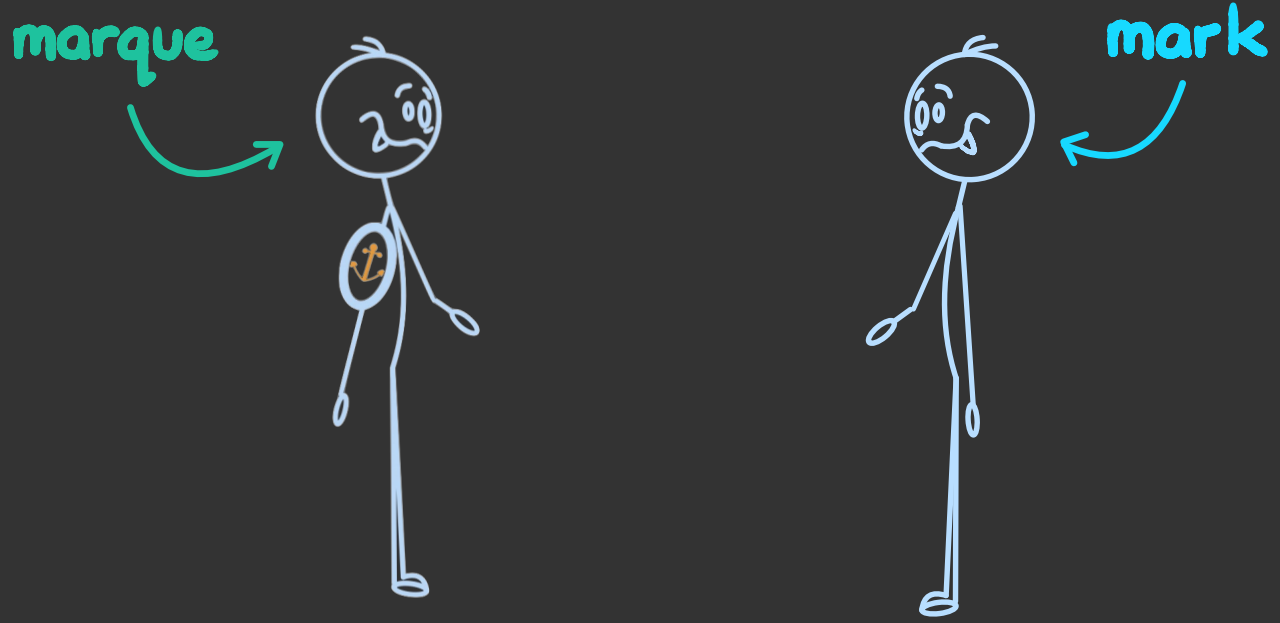
nice tattoo mark

as well as people who refer to me as mark

but if i were to clone myself

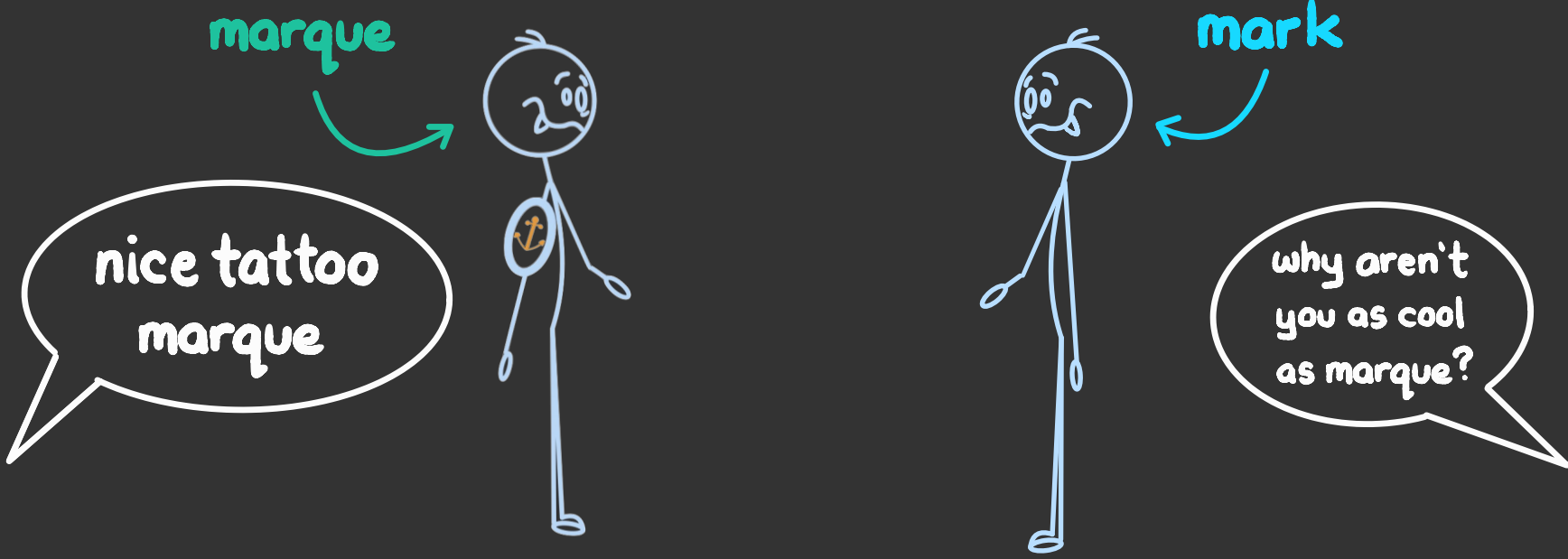


and then my clone got a tattoo



i would not suddenly acquire

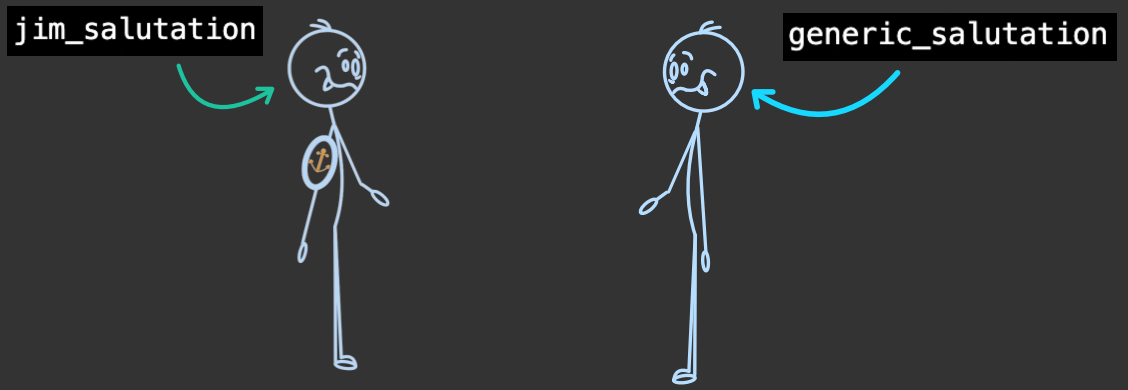
the same tattoo



```
In [1]: generic_salutation = ["Dear", "[blank]", ","]

In [2]: jim_salutation = ["Dear", "[blank]", ","]

In [3]: jim_salutation[1] = "Jim Bern"

In [4]: jim_salutation
Out[4]: ['Dear', 'Jim Bern', ',']

In [5]: generic_salutation
Out[5]: ['Dear', '[blank]', ',']
```

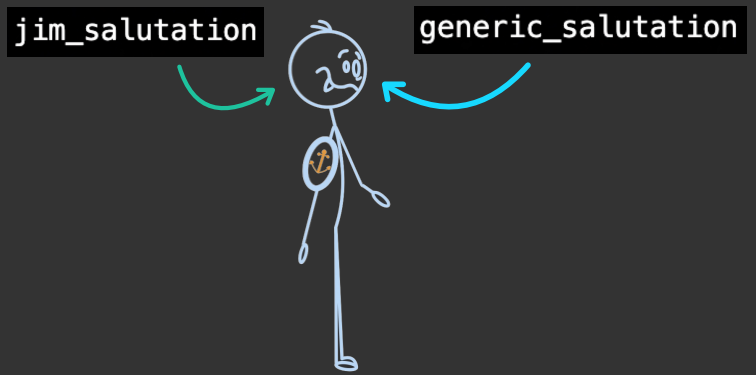
← changed

← unchanged



```
In [1]: generic_salutation = ["Dear", "[blank]", ","]

In [2]: jim_salutation = generic_salutation

In [3]: jim_salutation
Out[3]: ['Dear', '[blank]', ',']

In [4]: jim_salutation[1] = "Jim Bern"

In [5]: jim_salutation
Out[5]: ['Dear', 'Jim Bern', ',']

In [6]: generic_salutation
Out[6]: ['Dear', 'Jim Bern', ',']
```

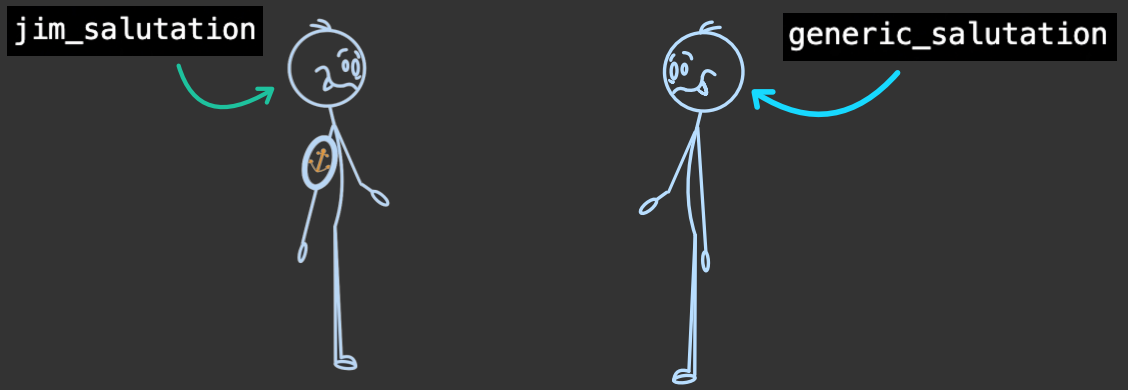
changed



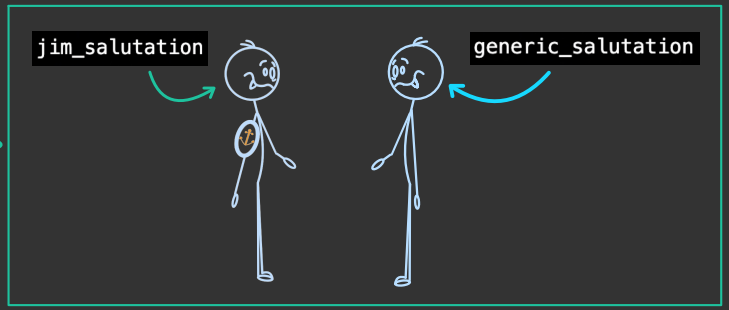
```
In [1]: generic_salutation = ["Dear", "[blank]", ","]

In [2]: jim_salutation = generic_salutation[0:]

In [3]: jim_salutation
Out[3]: ['Dear', '[blank]', ',']

In [4]: jim_salutation[1] = "Jim Bern"

In [5]: jim_salutation
Out[5]: ['Dear', 'Jim Bern', ',']

In [6]: generic_salutation
Out[6]: ['Dear', '[blank]', ',']
```

changed

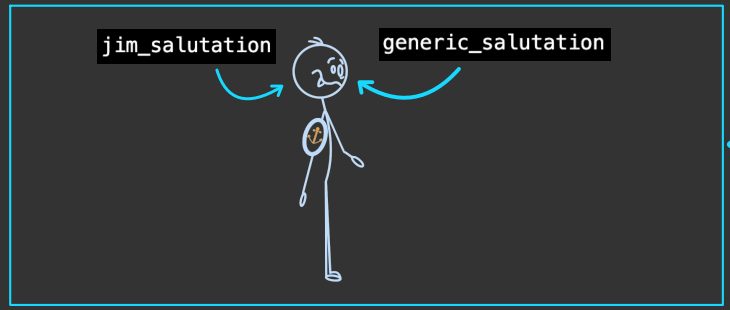
unchanged



```
In [1]: generic_salutation = ["Dear", "[blank]", ","]
In [2]: jim_salutation = generic_salutation[0:] # makes a clone
In [3]: jim_salutation = generic_salutation     # just gives the list an alternate name
```



cloning



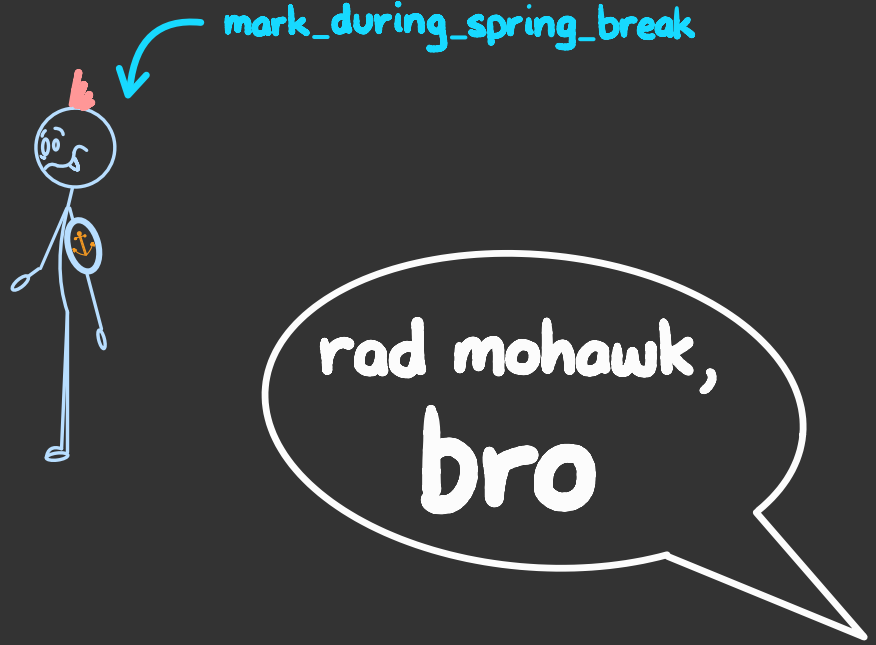
aliasing

because we can change the contents of a list after we've created it, list is referred to as

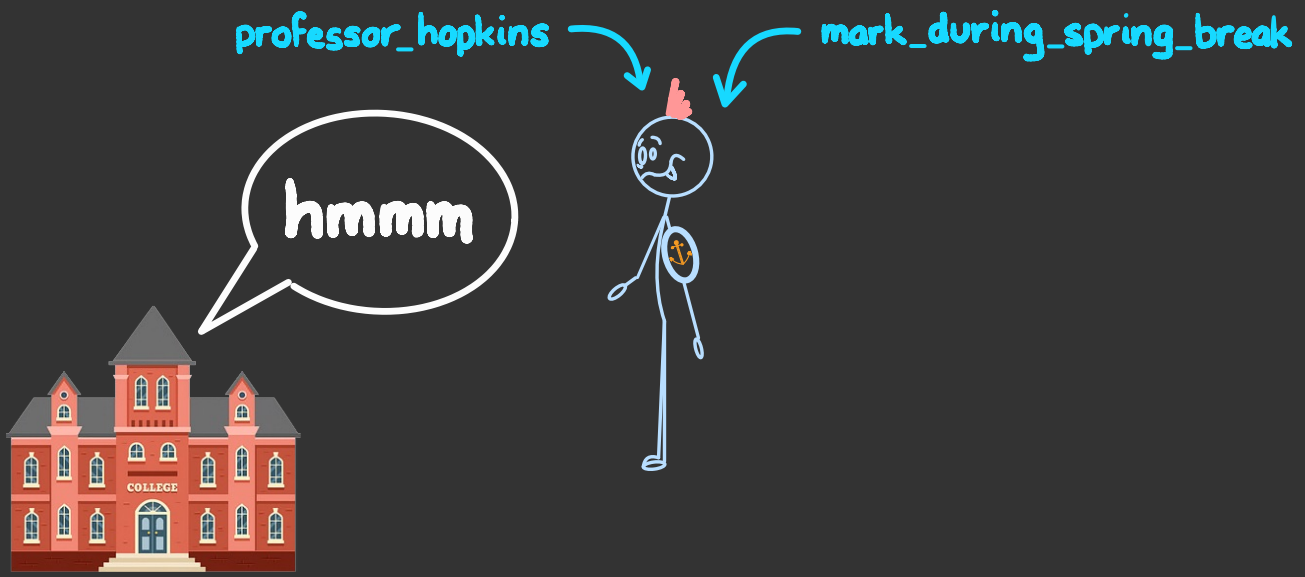
a mutable type

mutable types sometimes have

undesired consequences



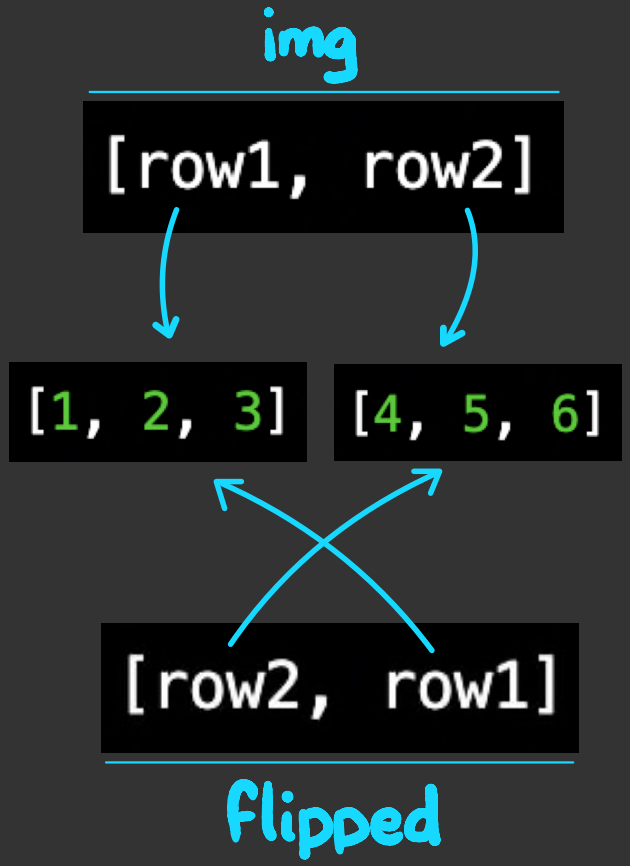
especially if we forget about other contexts in which an entity is being used



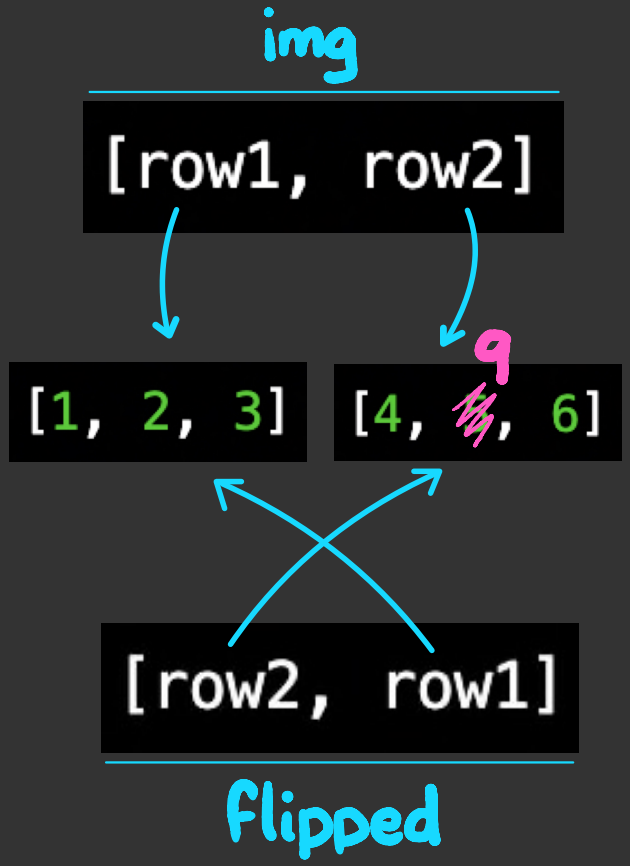
```
In [1]: row1 = [1, 2, 3]

In [2]: row2 = [4, 5, 6]

In [3]: img = [row1, row2]

In [4]: flipped = img[::-1]

In [5]: img
Out[5]: [[1, 2, 3], [4, 5, 6]]

In [6]: flipped
Out[6]: [[4, 5, 6], [1, 2, 3]]

In [7]: flipped[0][1] = 9

In [8]: flipped
Out[8]: [[4, 9, 6], [1, 2, 3]]

In [9]: img
Out[9]: [[1, 2, 3], [4, 9, 6]]
```


why does this happen?

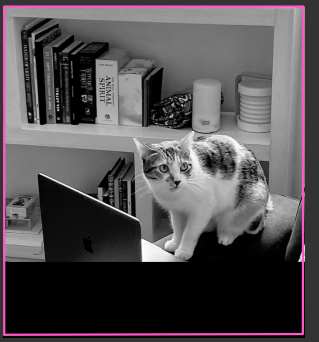
```
In [1]: row1 = [1, 2, 3]

In [2]: row2 = [4, 5, 6]

In [3]: img = [row1, row2]

In [4]: flipped = img[::-1]

In [5]: img
Out[5]: [[1, 2, 3], [4, 5, 6]]

In [6]: flipped
Out[6]: [[4, 5, 6], [1, 2, 3]]
```



```
In [1]: row1 = [1, 2, 3]

In [2]: row2 = [4, 5, 6]

In [3]: img = [row1, row2]

In [4]: flipped = img[::-1]

In [5]: img
Out[5]: [[1, 2, 3], [4, 5, 6]]

In [6]: flipped
Out[6]: [[4, 5, 6], [1, 2, 3]]

In [7]: flipped[0][1] = 9

In [8]: flipped
Out[8]: [[4, 9, 6], [1, 2, 3]]

In [9]: img
Out[9]: [[1, 2, 3], [4, 9, 6]]
```



```
In [1]: from imageutil import *

In [2]: img = read_image("images/cat.jpg")

In [3]: flipped = img[::-1]

In [4]: for row in range(0, 200):
   ...:     for col in range(0, 794):
   ...:         flipped[row][col] = 0
   ...:

In [5]: show_image(flipped)

In [6]: show_image(img)
```



this also changes the original image

because mutability can be dangerous,
some expression types cannot be changed

```
In [1]: generic_salutation = "Dear X,"

In [2]: generic_salutation[5] = "M"
---------------------------------------------------------------
TypeError                                 Traceback (most
Cell In[2], line 1
----> 1 generic_salutation[5] = "M"

TypeError: 'str' object does not support item assignment
```

For instance, strings are immutable

python even provides an immutable version of a list called a tuple

```
In [1]: group_f = ["Belgium", "Canada", "Croatia", "Morocco"]

In [2]: group_f[1] = "Italy"

In [3]: group_f
Out[3]: ['Belgium', 'Italy', 'Croatia', 'Morocco']

In [4]: group_e = ("Japan", "Spain", "Germany", "Costa Rica")

In [5]: group_e[1] = "Portugal"
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[5], line 1
----> 1 group_e[1] = "Portugal"

TypeError: 'tuple' object does not support item assignment

In [6]: group_e
Out[6]: ('Japan', 'Spain', 'Germany', 'Costa Rica')
```

tuples can be converted to lists, and vice versa

```
In [1]: group_f = ["Belgium", "Canada", "Croatia", "Morocco"]

In [2]: group_f_immutable = tuple(group_f)

In [3]: group_f[1] = "Italy"

In [4]: group_f_immutable[1] = "Italy"
---------------------------------------------------------------------------
TypeError
Cell In[4], line 1
----> 1 group_f_immutable[1] = "Italy"

TypeError: 'tuple' object does not support item assignment

In [5]: group_f
Out[5]: ['Belgium', 'Italy', 'Croatia', 'Morocco']

In [6]: group_f_immutable
Out[6]: ('Belgium', 'Canada', 'Croatia', 'Morocco')
```

expressions so far

type	example
int	7
float	3.14
str	"williams"
bool	3 > 4
function	abs
NoneType	print("hi")
range	range(1,5)
list	["mh","rb","jb","rb"]
dict	{"a":1,"b":2,"c":3}
set	{"mh","rb","jb"}
tuple	("mh","rb","jb","rb")